ROS 2 安装与话题通信实验报告

——基于 Windows 11 + VMware 虚拟机中 Docker 容器的 ROS 2 Foxy 实践

实验课程： ROS 2 机器人操作系统

宿主机： Windows 11

虚拟机： VMware Workstation + Ubuntu 22.04 LTS

容器环境： Docker + Ubuntu 20.04 + ROS 2 Foxy

姓　名： _____

学　号： _____

实验日期： 2026 年 10 月 27 日

一、实验目的

（1）在 Ubuntu 环境（Docker 容器）中完成 ROS 2 Foxy 的部署与验证，掌握 ROS 2 发行版与 Ubuntu 版本的对应关系，以及 Docker 图形化容器的配置方法。

（2）通过官方示例节点 talker / listener 验证安装结果，理解节点、话题、消息等核心概念，掌握 ros2 node、ros2 topic 等命令行工具。

（3）掌握 ROS 2 工作空间与功能包的创建流程，独立编写 C++ 发布者与订阅者节点，完成编译、运行与通信验证。

（4）利用 turtlesim 小海龟仿真与 rqt_graph 可视化工具直观理解计算图结构，并对话题发布频率进行定量分析。

二、实验环境与说明

本实验在 Windows 11 主机上通过 VMware Workstation 启动 Ubuntu 22.04 LTS 虚拟机，再在虚拟机中运行 Docker 容器，ROS 2 Foxy 安装于容器内部。所有终端操作、编译与运行均在容器中完成，虚拟机与宿主机的系统环境保持干净。

> 说明：ROS 2 的各发行版与 Ubuntu 版本严格绑定。Foxy 的官方二进制包只支持 Ubuntu 20.04（Focal），因此在 Ubuntu 22.04 虚拟机中不能直接 apt 安装 ROS 2 Foxy，本实验采用 Docker 容器提供 Ubuntu 20.04 + ROS 2 Foxy 的运行环境，宿主机与虚拟机仅负责提供 Docker 与图形显示转发。

项目	配置
宿主机操作系统	Windows 11
虚拟化平台	VMware Workstation
虚拟机操作系统	Ubuntu 22.04 LTS（Jammy），x86_64
容器平台	Docker（容器名 ros2_foxy_dev）
容器内操作系统	Ubuntu 20.04 LTS（Focal）
ROS 2 发行版	Foxy Fitzroy（LTS）
ROS 2 安装路径	/opt/ros/foxy
已安装 ROS 包数量	258 个（ros2 pkg list 统计）
编译工具	colcon + ament_cmake
工作空间	/root/ros2_ws_cls
编程语言	C++（rclcpp）、Python 3.8（rclpy）

三、ROS 2 Foxy 环境准备

3.1 发行版与 Ubuntu 版本对应关系

ROS 2 每个发行版都有唯一对应的 Ubuntu 版本，安装前必须先确认二者匹配，否则会出现依赖库版本冲突、Python 模块无法加载等问题。本次实验最初在 Ubuntu 22.04 虚拟机中直接执行 source /opt/ros/humble/setup.bash，由于系统中只有 Foxy 环境，提示 “No such file or directory”，因此改为在容器中使用 Foxy 对应的 Ubuntu 20.04。

ROS 2 发行版	对应 Ubuntu	代号	支持状态
Foxy Fitzroy	Ubuntu 20.04	Focal	LTS，本实验使用
Humble Hawksbill	Ubuntu 22.04	Jammy	LTS
Jazzy Jalisco	Ubuntu 24.04	Noble	当前 LTS

3.2 启动 Docker 容器

虚拟机中已安装 Docker，并存在名为 ros2_foxy_dev、内部预装 ROS 2 Foxy 的容器。启动时需把宿主机的 X11 套接字与 DISPLAY 变量传入容器，否则 turtlesim、rqt_graph 等图形程序无法打开窗口；同时使用 --net=host 让容器与主机共用网络，便于多终端之间的 DDS 自动发现。

```
# 虚拟机（宿主机侧）：允许容器内的 root 用户访问本机 X 服务
xhost +SI:localuser:root

# 启动已有容器并进入交互式 shell
sudo docker start -ai ros2_foxy_dev

# 如需新建容器，等价的启动命令为：
# sudo docker run -it --net=host \
#   -e DISPLAY=$DISPLAY \
#   -v /tmp/.X11-unix:/tmp/.X11-unix \
#   -v ~/ros2_ws_cls:/root/ros2_ws_cls \
#   --name ros2_foxy_dev <ros-foxy 镜像> bash
```

容器启动后命令提示符变为 root@gjy-virtual-machine:/#，说明已进入容器内部；此时所有 ros2 命令都运行在容器中，与虚拟机的 Ubuntu 22.04 相互隔离。

3.3 加载 ROS 2 环境变量

ROS 2 的可执行文件、包索引与环境变量由 setup.bash 注入。进入容器后必须先 source 对应发行版的脚本，且路径必须与实际安装的发行版一致。

```
source /opt/ros/foxy/setup.bash
printenv ROS_DISTRO          # 期望输出 foxy
```

若误写成 /opt/ros/humble/setup.bash，会提示 No such file or directory，这与是否安装成功无关，而是发行版路径写错，属于本实验最先遇到的问题。

3.4 安装结果验证

通过发行版变量、软件包索引、消息接口定义三个层面验证环境是否完整：

```
$ printenv ROS_DISTRO
foxy
$ ros2 pkg list | wc -l
258
$ ros2 interface show std_msgs/msg/String
# This was originally provided as an example message.
# It is deprecated as of Foxy
# It is recommended to create your own semantically meaningful message.
# However if you would like to continue using this please use the equivalent in
example_msgs.

string data
```

ROS_DISTRO 为 foxy 说明环境变量生效；258 个包说明 Foxy 桌面组件完整；接口定义正确输出字段 data（类型 string），证明消息生成代码与类型支持系统工作正常。输出中的 deprecated 注释是 Foxy 版本对示例消息的提示，并不影响 std_msgs/msg/String 的正常使用。

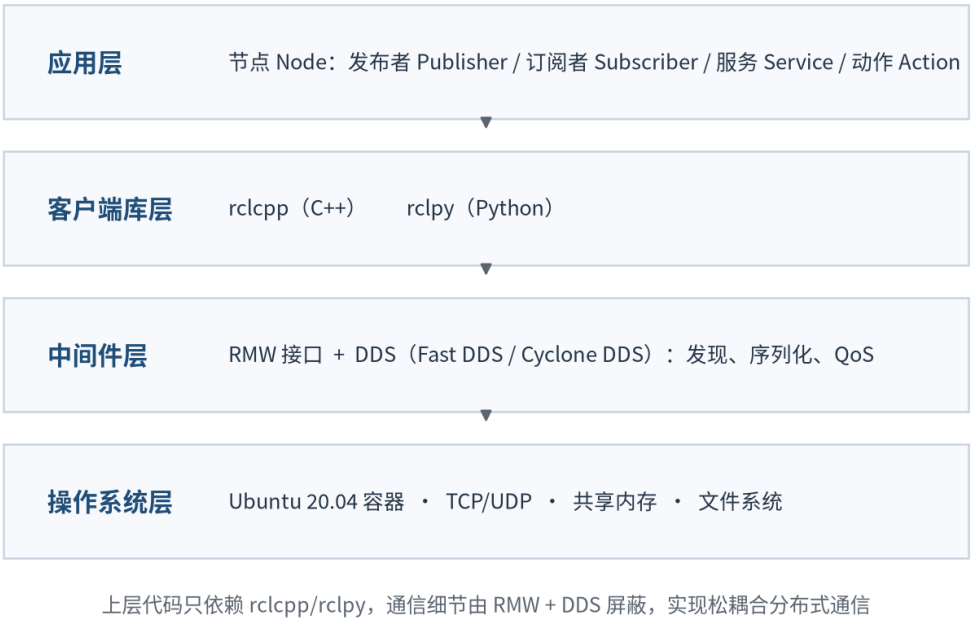


图 1 ROS 2 分层架构与本次实验所处层次

四、官方示例节点验证

环境就绪后，使用 demo_nodes_cpp / demo_nodes_py 中的 talker 与 listener 验证节点通信。两个节点分别运行在两个容器终端中，通过 /chatter 话题以 std_msgs/msg/String 通信。

```
# 终端 1（容器内）：C++ 发布者
source /opt/ros/foxy/setup.bash
ros2 run demo_nodes_cpp talker

# 终端 2（容器内）：Python 订阅者
source /opt/ros/foxy/setup.bash
ros2 run demo_nodes_py listener
```

发布者严格以 1 Hz 的节奏递增发布，订阅者逐条接收，实测连续运行期间未出现丢包或异常退出。



图 2 talker 节点以 1 Hz 发布 /chatter 话题（Hello World: 1~21）



图 3 listener 节点同步接收话题消息（Hello World: 18~35）

在第三个终端中进一步观察计算图，节点与话题信息完全符合预期：

```
$ ros2 node list
/listener
/talker

$ ros2 topic list
/chatter
/parameter_events
/rosout

$ ros2 topic info /chatter
Type: std_msgs/msg/String
Publisher count: 1
Subscription count: 1
```

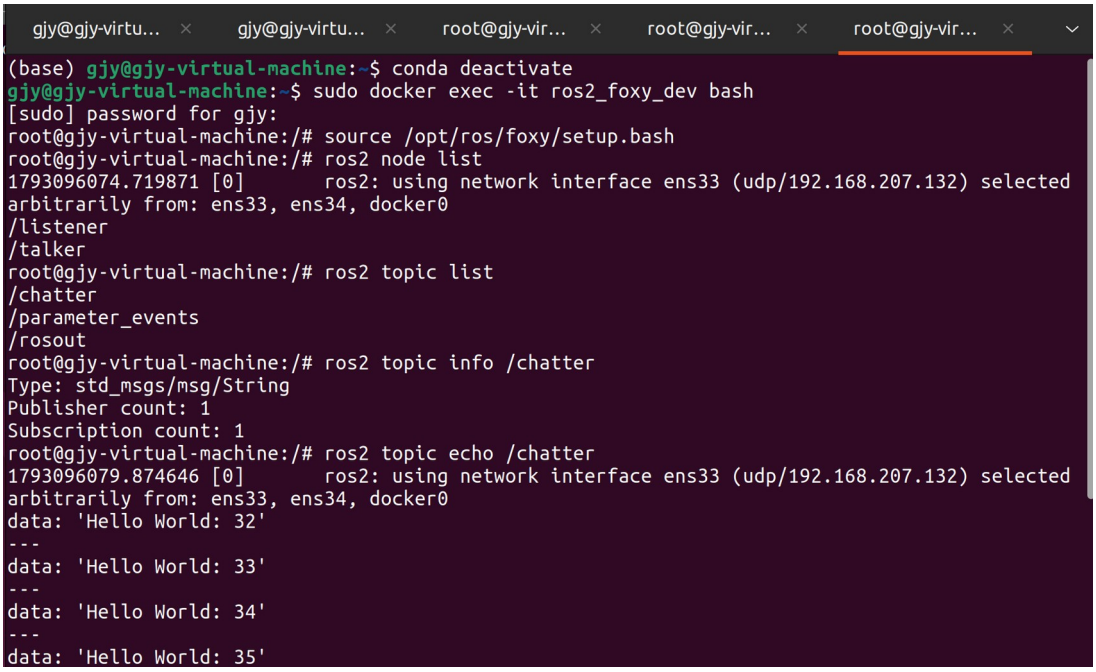


图 4 命令行查看节点、话题与发布/订阅计数，并回显消息内容

其中 /parameter_events 与 /rosout 是每个节点默认创建的公共服务话题，/chatter 才是本示例的业务话题；Publisher count 与 Subscription count 均为 1，说明一条话题上已正确建立“一发布一订阅”的连接。

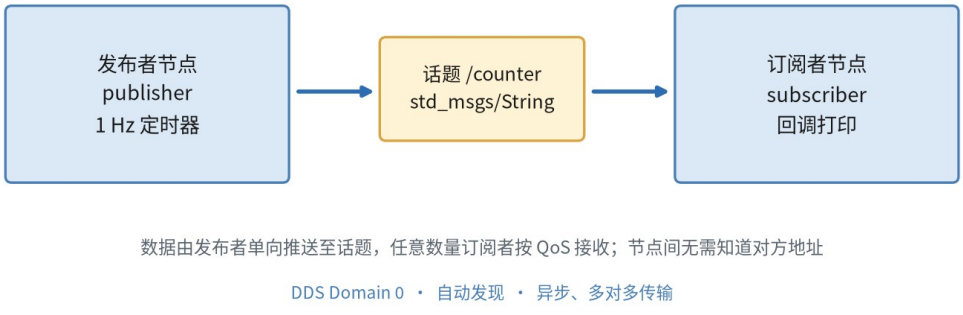


图 5 基于话题的发布/订阅通信模型

五、小海龟仿真体验

turtlesim 是 ROS 2 官方提供的二维仿真器，常用于验证话题、服务与参数机制。在容器中启动仿真节点与键盘控制节点后，即可用方向键驱动小海龟运动。

```
# 终端 1（容器内）
source /opt/ros/foxy/setup.bash
ros2 run turtlesim turtlesim_node

# 终端 2（容器内）
source /opt/ros/foxy/setup.bash
ros2 run turtlesim turtle_teleop_key
```

键盘控制节点启动后提示 “Reading from keyboard / Use arrow keys to move the turtle”，此时焦点切到该终端即可通过方向键控制。图 6 为使用方向键与旋转键操作后留下的运动轨迹。



图 6 通过 turtle_teleop_key 键盘控制小海龟运动

为进一步验证速度指令与运动学的关系，直接用话题发布 Twist 速度指令。Twist 的线速度 linear.x 与角速度 angular.z 决定转弯半径 r = v / ω：令 v = 0.85、ω = 1.0，小海龟应沿半径约 0.85 的圆运动。

```
ros2 topic pub -r 20 /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.85}, angular: {z: 1.0}}"
```

```
root@gjy-virtual-machine:/# ros2 topic pub -r 20 /turtle1/cmd_vel geometry_msgs/msg/Twist "{lin
ear: {x: 0.85}, angular: {z: 1.0}}"
1793095126.216153 [0]       ros2: using network interface ens33 (udp/192.168.207.132) selected
arbitrarily from: ens33, ens34, docker0
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.85, y=0.0, z=0.0),
angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.85, y=0.0, z=0.0),
angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.85, y=0.0, z=0.0),
angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #4: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.85, y=0.0, z=0.0),
angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #5: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.85, y=0.0, z=0.0),
angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #6: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.85, y=0.0, z=0.0)
```

图 7 以 20 Hz 持续发布 Twist 速度指令

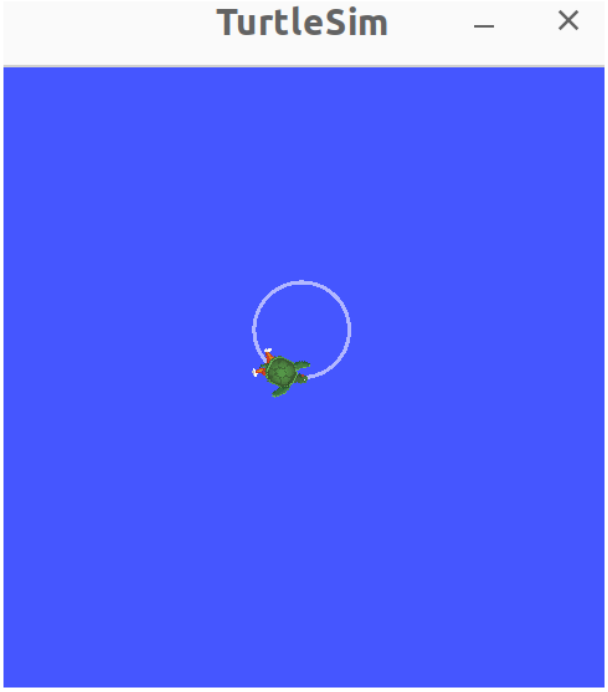


图 8 小海龟沿近似圆形轨迹运动，验证速度指令与运动学关系

容器内终端提示 “using network interface ens33 (udp/192.168.207.132) selected arbitrarily from: ens33, ens34, docker0”，说明 DDS 已选定通信网卡，多终端之间可正常互相发现。

六、话题通信编程实验

6.1 创建工作空间与功能包

ROS 2 采用工作空间组织源码与编译产物，src 存放源码，build/install/log 由 colcon 生成。在容器内使用 ros2 pkg create 创建 C++ 功能包，并声明对 rclcpp、std_msgs 的依赖。

```
mkdir -p ~/ros2_ws_cls/src
cd ~/ros2_ws_cls/src
ros2 pkg create --build-type ament_cmake pub_sub_demo \
  --dependencies rclcpp std_msgs
```

6.2 发布者节点实现

发布者节点继承 rclcpp::Node，创建名为 counter 的 String 话题，并注册 1 秒周期的定时器，在回调中递增计数并发布。代码位于 src/publisher.cpp：

```
#include <chrono>
#include <memory>
#include <string>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

class PublisherNode : public rclcpp::Node
{
public:
  PublisherNode() : Node("publisher_node"), count_(0)
  {
    publisher_ = this->create_publisher<std_msgs::msg::String>("counter", 10);
    timer_ = this->create_wall_timer(
      1s, std::bind(&PublisherNode::timer_callback, this));
  }

private:
  void timer_callback()
  {
    auto message = std_msgs::msg::String();
    message.data = "counter: " + std::to_string(count_++);
    RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
    publisher_->publish(message);
  }

  rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
  rclcpp::TimerBase::SharedPtr timer_;
  size_t count_;
};

int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<PublisherNode>());
  rclcpp::shutdown();
  return 0;
}
```

6.3 订阅者节点实现

订阅者节点订阅同一话题 counter，在回调中打印接收到的内容。代码位于 src/subscriber.cpp：

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

class SubscriberNode : public rclcpp::Node
{
public:
  SubscriberNode() : Node("subscriber_node")
  {
    subscription_ = this->create_subscription<std_msgs::msg::String>(
      "counter", 10,
      std::bind(&SubscriberNode::topic_callback, this, std::placeholders::_1));
  }

private:
  void topic_callback(const std_msgs::msg::String::SharedPtr msg) const
  {
    RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg->data.c_str());
  }

  rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<SubscriberNode>());
  rclcpp::shutdown();
  return 0;
}
```

6.4 配置 CMakeLists.txt 并编译

需要在自动生成的 CMakeLists.txt 中、ament_package() 之前补充可执行文件的编译与安装规则，否则 colcon 只生成空包，ros2 run 会提示找不到可执行文件。

```
add_executable(publisher src/publisher.cpp)
ament_target_dependencies(publisher rclcpp std_msgs)

add_executable(subscriber src/subscriber.cpp)
ament_target_dependencies(subscriber rclcpp std_msgs)

install(TARGETS
  publisher
  subscriber
  DESTINATION lib/${PROJECT_NAME}
)
```

在工作空间根目录执行编译，本次编译（含消息依赖处理与链接）耗时 34.2 秒：

```
$ cd ~/ros2_ws_cls
$ colcon build --packages-select pub_sub_demo
Starting >>> pub_sub_demo
[Processing: pub_sub_demo]
Finished <<< pub_sub_demo [34.2s]

Summary: 1 package finished [34.9s]
```

```
root@gjy-virtual-machine:~/ros2_ws_cls# colcon build --packages-select pub_sub_demo
Starting >>> pub_sub_demo
[Processing: pub_sub_demo]
Finished <<< pub_sub_demo [34.2s]

Summary: 1 package finished [34.9s]
root@gjy-virtual-machine:~/ros2_ws_cls#
```

图 9 colcon 编译 pub_sub_demo 功能包成功

6.5 运行与通信验证

分别启动发布者与订阅者（各自 source install/setup.bash），观察输出与计算图。

```
# 终端 1
cd ~/ros2_ws_cls && source install/setup.bash
ros2 run pub_sub_demo publisher

# 终端 2
cd ~/ros2_ws_cls && source install/setup.bash
ros2 run pub_sub_demo subscriber
```

```
root@gjy-virtual-machine:~/ros2_ws_cls# source install/setup.bash
root@gjy-virtual-machine:~/ros2_ws_cls# ros2 run pub_sub_demo publisher
1793098285.423290 [0]  publisher: using network interface ens33 (udp/192.168.207.132) selected
arbitrarily from: ens33, ens34, docker0
[INFO] [1793098286.455287016] [publisher_node]: Publishing: 'counter: 0'
[INFO] [1793098287.455489917] [publisher_node]: Publishing: 'counter: 1'
[INFO] [1793098288.455463065] [publisher_node]: Publishing: 'counter: 2'
[INFO] [1793098289.455311313] [publisher_node]: Publishing: 'counter: 3'
[INFO] [1793098290.455403777] [publisher_node]: Publishing: 'counter: 4'
[INFO] [1793098291.455613589] [publisher_node]: Publishing: 'counter: 5'
[INFO] [1793098292.455924394] [publisher_node]: Publishing: 'counter: 6'
[INFO] [1793098293.455261375] [publisher_node]: Publishing: 'counter: 7'
[INFO] [1793098294.456480588] [publisher_node]: Publishing: 'counter: 8'
[INFO] [1793098295.456243435] [publisher_node]: Publishing: 'counter: 9'
[INFO] [1793098296.455585203] [publisher_node]: Publishing: 'counter: 10'
[INFO] [1793098297.455584247] [publisher_node]: Publishing: 'counter: 11'
[INFO] [1793098298.455619194] [publisher_node]: Publishing: 'counter: 12'
[INFO] [1793098299.455759220] [publisher_node]: Publishing: 'counter: 13'
^C[INFO] [1793098299.830368637] [rclcpp]: signal_handler(signal_value=2)
root@gjy-virtual-machine:~/ros2_ws_cls#
```

图 10 自定义 publisher_node 以 1 Hz 发布 counter: 0~13

```
root@gjy-virtual-machine:~/ros2_ws_cls# source install/setup.bash
root@gjy-virtual-machine:~/ros2_ws_cls# ros2 run pub_sub_demo subscriber
1793098287.825596 [0] subscriber: using network interface ens33 (udp/192.168.207.132) selected
arbitrarily from: ens33, ens34, docker0
[INFO] [1793098288.456601231] [subscriber_node]: I heard: 'counter: 2'
[INFO] [1793098289.455956320] [subscriber_node]: I heard: 'counter: 3'
[INFO] [1793098290.456529805] [subscriber_node]: I heard: 'counter: 4'
[INFO] [1793098291.456358119] [subscriber_node]: I heard: 'counter: 5'
[INFO] [1793098292.456861746] [subscriber_node]: I heard: 'counter: 6'
[INFO] [1793098293.456840878] [subscriber_node]: I heard: 'counter: 7'
[INFO] [1793098294.458502717] [subscriber_node]: I heard: 'counter: 8'
[INFO] [1793098295.458141684] [subscriber_node]: I heard: 'counter: 9'
[INFO] [1793098296.456382524] [subscriber_node]: I heard: 'counter: 10'
[INFO] [1793098297.457011235] [subscriber_node]: I heard: 'counter: 11'
[INFO] [1793098298.456542298] [subscriber_node]: I heard: 'counter: 12'
[INFO] [1793098299.457523700] [subscriber_node]: I heard: 'counter: 13'
^C[INFO] [1793098301.989347613] [rclcpp]: signal_handler(signal_value=2)
root@gjy-virtual-machine:~/ros2_ws_cls#
```

图 11 自定义 subscriber_node 接收 counter: 2~13，与发布端逐条对应

发布端日志时间戳由 1793098286.455 至 1793098299.755，共 14 条消息、跨度 13.300 秒，平均间隔约 1.000 秒；订阅者晚约 2 秒启动，因此首条收到的是 counter: 2，此后逐条同步、无丢失，说明话题通信已正常建立。命令行侧的验证结果如图 12 所示：节点列表为 /publisher_node 与

/subscriber_node，话题列表中业务话题为 /counter，其发布与订阅计数均为 1，ros2 topic echo 依次输出递增的计数内容。

```
/publisher_node
/subscriber_node
root@gjy-virtual-machine:/# ros2 topic list
/counter
/parameter_events
/rosout
root@gjy-virtual-machine:/# ros2 topic info /counter
Type: std_msgs/msg/String
Publisher count: 1
Subscription count: 1
root@gjy-virtual-machine:/# ros2 topic echo /counter
1793098500.467922 [0]       ros2: using network interface ens33 (udp/192.168.207.132) selected
arbitrarily from: ens33, ens34, docker0
data: 'counter: 45'
---
data: 'counter: 46'
---
data: 'counter: 47'
---
data: 'counter: 48'
```

图 12 命令行确认 /counter 的节点、话题、发布订阅计数与消息内容

6.6 发布频率定量分析

为进一步评价定时器精度与通信稳定性，使用 ros2 topic hz 对 /counter 进行统计，运行约 60 秒（统计窗口累积至 57 条）后按 Ctrl+C 结束，结果如下：

统计量	数值	说明
平均频率	1.000 Hz	设定值 1 Hz
平均周期	约 1.000 s	与定时器设定一致
最小间隔	0.998 s	受系统调度影响
最大间隔	1.002 s	受系统调度影响
标准差	0.00103 s（≈1.03 ms）	抖动约 0.1%
统计窗口	57 条	窗口逐步累积后趋于稳定

平均频率与设定值 1 Hz 完全一致，标准差仅约 1.03 ms，说明 rclcpp 的 wall timer 在容器环境中仍具有很高的定时精度，且 DDS 在同一主机内的传输时延极小，未出现消息堆积或丢失。需要说明的是，ros2 topic hz 采用滑动窗口统计，窗口较小时（window: 2）数值波动较大，待窗口累积到数十条后读数才趋于稳定，这与实际发布精度无关。



图 13 rqt_graph 显示 publisher_node 与 subscriber_node 通过 /counter 连接

七、实验过程中遇到的问题与解决

实验过程中主要遇到发行版路径、容器图形转发与工程配置三类问题，汇总如下。

序号	现象	原因分析	解决办法
1	ros2: command not found	新终端未加载 ROS 2 环境变量	执行 source /opt/ros/foxy/setup.bash 后再使用 ros2 命令
2	source /opt/ros/humble/setup.bash 提示 No such file or directory	本环境安装的是 Foxy，路径应为 /opt/ros/foxy	改用 source /opt/ros/foxy/setup.bash，并 printenv ROS_DISTRO 确认
3	终端前缀出现 (base)，ros2 命令找不到	conda base 环境修改了 PATH，遮挡系统命令	先 conda deactivate，再进入容器操作
4	容器内启动 turtlesim 报 Authorization required / could not connect to display :0	容器无法直接访问虚拟机 X 服务，且未传递 DISPLAY	虚拟机中执行 xhost +SI:localuser:root，启动容器时传 -e DISPLAY 与 -v /tmp/.X11-unix
5	colcon build 成功但 ros2 run 提示找不到可执行文件	CMakeLists.txt 缺少 install(TARGETS ...) 规则	补充 add_executable 与 install 规则后重新编译
6	rqt_graph 启动时提示 XDG_RUNTIME_DIR not set	容器内没有用户 runtime 目录	仅为警告，不影响使用；可 export XDG_RUNTIME_DIR=/tmp/runtime-root 消除
7	std_msgs/msg/String 接口输出带 deprecated 注释	Foxy 版本对示例消息的提示信息	不影响通信，示例仍可正常使用该类型
8	ros2 topic hz 初期读数波动较大	滑动窗口样本数太少（window: 2）	持续运行使窗口累积到数十条后再读数

其中第 1~4 项属于环境配置问题，第 5 项是 ROS 2 工程配置中最容易遗漏的一步，第 6~8 项属于提示或统计方法问题，不影响实验结果。

八、实验结论

（1）在 Windows 11 + VMware Workstation + Ubuntu 22.04 的环境中，通过 Docker 容器部署了 Ubuntu 20.04 + ROS 2 Foxy，ROS_DISTRO 为 foxy，ros2 pkg list 共 258 个包，消息接口 std_msgs/msg/String 定义正确，环境完整可用。

（2）官方 talker / listener 示例运行正常，/chatter 话题的发布与订阅计数均为 1，节点列表、话题列表与消息回显结果一致，说明 DDS 通信层与命令行工具链工作正常。

（3）独立完成 pub_sub_demo 功能包：从工作空间创建、C++ 发布者与订阅者编写、CMakeLists.txt 配置到 colcon 编译（34.2 s）全流程跑通，/counter 话题实现一发布一订阅通信。

（4）频率统计显示平均频率 1.000 Hz、标准差约 1.03 ms，定时精度与容器内传输稳定性良好，满足常规机器人应用的实时性要求。

（5）turtlesim 与 rqt_graph 从可视化角度印证了话题通信模型：节点通过具名话题解耦，发布者与订阅者无需知晓对方身份即可交换消息，这正是 ROS 2 分布式架构的核心优势。

通过本次实验，掌握了 ROS 2 的安装验证与命令行使用，理解了“节点—话题—消息”这一基本抽象如何支撑松耦合的机器人软件系统；同时也认识到 ROS 2 发行版与 Ubuntu 版本强绑定，以及 Docker 环境中图形转发与网络模式配置的重要性。实验中最初出现的 “No such file or directory” 并非安装失败，而是发行版路径与实际环境不匹配，排查时应先确认 ROS_DISTRO 与 /opt/ros 下的实际目录，再检查环境变量。

附录 关键命令清单

阶段	命令	作用
授权	xhost +SI:localuser:root	允许容器访问本机 X 服务
容器	sudo docker start -ai ros2_foxy_dev	启动并进入 ROS 2 容器
环境	source /opt/ros/foxy/setup.bash	加载 ROS 2 Foxy 环境变量
验证	printenv ROS_DISTRO	确认发行版为 foxy
验证	ros2 pkg list | wc -l	统计已安装包数量
示例	ros2 run demo_nodes_cpp talker	启动官方发布者
建包	ros2 pkg create --build-type ament_cmake pub_sub_demo	创建 C++ 功能包
编译	colcon build --packages-select pub_sub_demo	编译指定功能包
运行	ros2 run pub_sub_demo publisher	运行自定义发布者
观察	ros2 topic echo /counter	查看话题消息内容
观察	ros2 topic hz /counter	统计话题发布频率
观察	rqt_graph	可视化节点与话题连接
仿真	ros2 run turtlesim turtlesim_node	启动小海龟仿真